

Sistema de reservas de salas de estudio — Informe

Autor (ficticio): Estudiante B — Grupo 1 **Materia:** Ingeniería de Software

1. Problema y requisitos

La biblioteca reserva sus salas de estudio con una hoja de cálculo, lo que provoca reservas repetidas y salas que quedan vacías. Se propone un sistema web para reservar y cancelar salas y ver la disponibilidad.

Requisitos:

- El estudiante puede reservar una sala libre en una fecha y hora.
- El estudiante puede cancelar su reserva.
- El personal puede ver la ocupación de las salas.
- El sistema debe ser rápido y fácil de usar.

(El último requisito es de tipo no funcional, pero no se especifica cuán rápido ni bajo qué condiciones.)

2. Arquitectura

El sistema tiene una interfaz web que se comunica con un backend, y el backend guarda los datos en una base de datos PostgreSQL. El backend expone una API REST con operaciones para crear, cancelar y consultar reservas. La interfaz web muestra el calendario de salas y permite seleccionar una franja para reservar. La lógica para evitar reservas duplicadas y para liberar salas no usadas está en el backend.

No se incluye un diagrama, pero los componentes principales son: interfaz web, API backend y base de datos.

3. Decisiones técnicas

- Se usó PostgreSQL porque es una base de datos relacional adecuada para datos estructurados como reservas y porque permite definir una restricción de unicidad que evita dos reservas en la misma sala y hora.
- Se eligió una API REST porque es un estándar conocido y facilita conectar la interfaz web con el backend.
- Para liberar las salas no confirmadas se usa una tarea programada que revisa periódicamente las reservas.

Las decisiones están justificadas en función del problema, aunque no se compararon alternativas en detalle.

4. Referencias

El diseño en capas busca separar la interfaz de la lógica de negocio, como recomienda la bibliografía de ingeniería de software para reducir el acoplamiento (Sommerville, 2016). La elección de garantizar la unicidad en la base de datos se apoya en buenas prácticas de manejo de concurrencia (Kleppmann, 2017).

- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly.
- Sommerville, I. (2016). *Software Engineering*. Pearson.

5. Conclusión

El sistema resuelve el problema de las reservas duplicadas y la ocupación de salas, y es más cómodo que la hoja de cálculo actual.